PRACTICAL - 04

Aim: Password strength (PasswordChecker.java + Driver.java)

Code Screenshot:

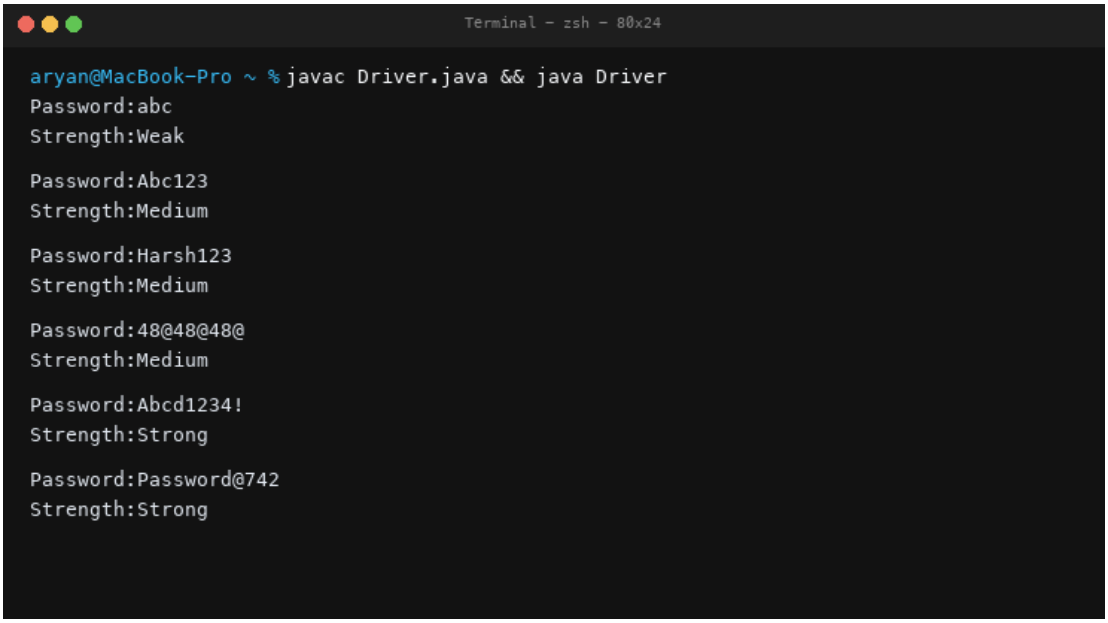
PasswordChecker.java

```java
public class PasswordChecker {
    public static String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8) score++;
        if (pw.matches(".*[A-Z].*")) score++;
        if (pw.matches(".*[0-9].*")) score++;
        if (pw.matches(".*[^a-zA-Z0-9].*")) score++;
        if (score <= 1) return "Weak";
        else if (score <= 3) return "Medium";
        return "Strong";
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] passwords = {
            "abc", "Abc123", "Harsh123", "48@48@48@", "Abcd1234!", "Password@742"
        };
        for (String pw : passwords) {
            String result = PasswordChecker.strength(pw);
            System.out.println("Password:" + pw);
            System.out.println("Strength:" + result);
            System.out.println();
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to validate passwords using multiple security rules.
- Understood the use of Regular Expressions (Regex) for pattern matching.
- Learned to use String methods such as length() and matches().
- Implemented password strength classification (Weak, Medium, Strong).
- Improved problem-solving and decision-making using conditional statements.

Aim: Chat-log filter (ChatFilter.java + Driver.java)

Code Screenshot:

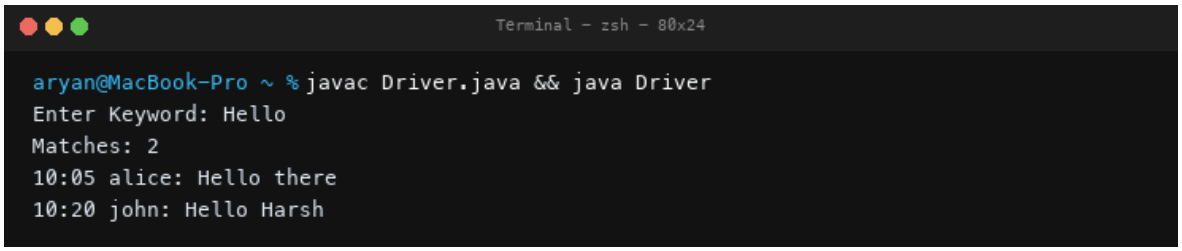
ChatFilter.java

```java
public class ChatFilter {
    public static String filter(String[] logs, String keyword) {
        int count = 0;
        StringBuilder report = new StringBuilder();
        for (String line : logs) {
            String[] parts = line.split(" ", 3);
            if (parts.length != 3) continue;
            String time = parts[0];
            String user = parts[1];
            String message = parts[2];
            if (message.toLowerCase().contains(keyword.toLowerCase())) {
                count++;
                report.append(time).append(" ").append(user).append(": ").append(message).append("\n");
            }
        }
        return "Matches: " + count + "\n" + report.toString();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:10 bob Good Morning",
            "WrongLine",
            "10:20 john Hello HarsH",
            "10:30 david Bye"
        };
        String keyword = "Hello";
        System.out.println("Enter Keyword: " + keyword);
        String result = ChatFilter.filter(logs, keyword);
        System.out.println(result);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to process and manipulate strings using split().
- Understood the use of StringBuilder for efficient string concatenation.
- Learned case-insensitive searching using toLowerCase() and contains().
- Implemented filtering of chat messages based on user input.
- Improved understanding of arrays, loops, and string processing techniques.